

PRACTICAL NO. 4

NAME : ATHARV SANJAY PATIL

BATCH : ET21

PRN : 202501070128

4.1.1. Pandas – series creation and manipulation

PROBLEM STATEMENT :

Write a Python program that takes a list of numbers from the user, creates a Pandas series from it, and then calculates the mean of even and odd numbers separately using the groupby and mean() operations.

Hint: You can group the Series based on whether each number is even or odd.

Input Format:

- The user should enter a list of numbers separated by space when prompted.

Output Format:

- The program should display the mean of even and odd numbers separately.
- Each mean value should be displayed with a label indicating whether it corresponds to even or odd numbers.

Refer to the sample test cases for better understanding regarding input and output format.

CODE :

```
import pandas as pd

# Take inputs from the user to create a list of numbers
numbers = list(map(int, input().split()))

# Create a Pandas series from the list of numbers
s = pd.Series(numbers)

# Grouping by even and odd numbers and calculating the mean
grouped = s.groupby(s % 2 == 0).mean()

# Display the mean of even and odd numbers with labels
grouped.index = ['Even' if is_even else 'Odd' for is_even in grouped.index]
print("Mean of even and odd numbers:")
print(grouped)
```

OUTPUT :

Average time
0.016 s
16.17 ms

Maximum time
0.042 s
42.00 ms

✓ 3 out of 3 shown test case(s) passed

✓ 3 out of 3 hidden test case(s) passed

✓ Test case 1 42 ms Debug

Expected output

1 2 3 4 5 6 7 8 9 10

Mean of even and odd numbers: 5.0

Odd: 5.0

Even: 6.0

dtype: float64

Actual output

1 2 3 4 5 6 7 8 9 10

Mean of even and odd numbers: 5.0

Odd: 5.0

Even: 6.0

dtype: float64

✓ Test case 2 12 ms Debug

Expected output

4 4 4 6 6 2 2 2 10 12 14 16

Mean of even and odd numbers: 6.833333

Even: 6.833333

dtype: float64

Actual output

4 4 4 6 6 2 2 2 10 12 14 16

Mean of even and odd numbers: 6.833333

Even: 6.833333

dtype: float64

✓ Test case 3 13 ms Debug

Expected output

1 5 7 9 11 13 15 17 111 21 25

Mean of even and odd numbers: 21.363636

Odd: 21.363636

Actual output

1 5 7 9 11 13 15 17 111 21 25

Mean of even and odd numbers: 21.363636

Odd: 21.363636

4.1.2. Dictionary to Dataframe

PROBLEM STATEMENT :

A dictionary of lists has been provided to you in the editor. Create a DataFrame from the dictionary of lists and perform the listed operations, then display the DataFrame before and after each manipulation.

Create the DataFrame:

- Convert the dictionary to a Pandas DataFrame.

Add a new row:

- Take inputs from the user for the new row data (name, age).
- Add the new row to the DataFrame.
- Display the DataFrame after adding the new row.

Modify a row:

- Modify a specific row by changing the age. Take the row index and new age value from the user.
- Display the DataFrame after modifying the row.

Delete a row:

- Take the row index to be deleted from the user.
- Remove the specified row.
- Display the DataFrame after deleting the row.

Add a new column:

- Add a column **Gender** with values taken from the user.
- Display the DataFrame after adding the new column.

Modify a column:

- Convert names to uppercase.
- Display the DataFrame after modifying the column.

Delete a column:

- Remove the **Age** column.
- Display the DataFrame after deleting the column.

CODE :

```
import pandas as pd

# Provided dictionary of lists
data = {'Name': ['Alice', 'Bob', 'Charlie'], 'Age': [25, 30, 35],}

# Convert the dictionary to a DataFrame
df = pd.DataFrame(data)

# Display the original DataFrame
print("Original DataFrame:")
print(df)

# Adding a new row
new_name = input("New name: ")
new_age = int(input("New age: "))
df.loc[len(df)] = [new_name, new_age]

# Display the DataFrame after adding a new row
print("After adding a row:\n",df)

# Modifying a row
row_to_modify = int(input("Index of row to modify: "))
new_age_value = int(input("New age: "))
df.at[row_to_modify, "Age"] = new_age_value

# Display the DataFrame after modifying a row
print("After modifying a row:")
print(df)
```

```
# Deleting a row

row_to_delete = int(input("Index of row to delete: "))

df = df.drop(index=row_to_delete).reset_index(drop=True)

# Display the DataFrame after deleting a row

print("After deleting a row:")

print(df)
```

```
# Adding a new column

genders = input("Enter genders separated by space: ").split()

df["Gender"] = genders
```

```
# Display the DataFrame after adding a new column

print("After adding a new column:")

print(df)
```

```
# Modifying a column

df["Name"] = df["Name"].str.upper()
```

```
# Display the DataFrame after modifying a column

print("After modifying a column:")

print(df)
```

```
# Deleting a column

df = df.drop(columns=["Age"])
```

```
# Display the DataFrame after deleting a column

print("After deleting a column:")

print(df)
```


OUTPUT :

Average time
0.136 s
136.00 ms

Maximum time
0.155 s
155.00 ms

1 out of 1 shown test case(s) passed

1 out of 1 hidden test case(s) passed

Test case 1 155 ms Debug

Expected output

Actual output

Original DataFrame:

.....Name Age

0 Alice 25

1 Bob 30

2 Charlie 35

Now name: Susan

Now age: 29

After adding a row:

.....Name Age

0 Alice 25

1 Bob 30

2 Charlie 35

3 Susan 29

Index of row to modify: 0

Now age: 27

After modifying a row:

.....Name Age

0 Alice 27

1 Bob 30

2 Charlie 35

3 Susan 29

Index of row to delete: 3

After deleting a row:

.....Name Age

0 Alice 27

1 Bob 30

2 Charlie 35

Enter genders separated by space: F M M

After adding a new column:

.....Name Age Gender

0 Alice 27 F

1 Bob 30 M

2 Charlie 35 M

After modifying a column:

.....Name Age Gender

0 ALICE 27 F

1 BOB 30 M

2 CHARLIE 35 M

After deleting a column:

.....Name Gender

0 ALICE F

1 BOB M

2 CHARLIE M

Original DataFrame:

.....Name Age

0 Alice 25

1 Bob 30

2 Charlie 35

Now name: Susan

Now age: 29

After adding a row:

.....Name Age

0 Alice 25

1 Bob 30

2 Charlie 35

3 Susan 29

Index of row to modify: 0

Now age: 27

After modifying a row:

.....Name Age

0 Alice 27

1 Bob 30

2 Charlie 35

3 Susan 29

Index of row to delete: 3

After deleting a row:

.....Name Age

0 Alice 27

1 Bob 30

2 Charlie 35

Enter genders separated by space: F M M

After adding a new column:

.....Name Age Gender

0 Alice 27 F

1 Bob 30 M

2 Charlie 35 M

After modifying a column:

.....Name Age Gender

0 ALICE 27 F

1 BOB 30 M

2 CHARLIE 35 M

After deleting a column:

.....Name Gender

0 ALICE F

1 BOB M

2 CHARLIE M

4.1.3. Student Information

PROBLEM STATEMENT :

Write a program to read a text file containing student information (name, age, and grade) using Pandas. Perform the following tasks:

- Display the first five rows of the data frame.
- Calculate the average age of the students (limit the average age up to 2 decimal places).
- Filter out the students who have a grade above a certain threshold (consider the threshold grade is **'B'**).

Note:

Refer to the displayed test cases for better understanding.

CODE :

```
import pandas as pd

# Read the text file into a DataFrame
file = input()
data = pd.read_csv(file, sep="\s+", header=None, names=["Name", "Age", "Grade"])

# write your code here..

# Display the first five rows of the data frame
print("First five rows:")
print(data.head())

# Calculate the average age of the students(limit the average age up to 2 decimal
places)
avg_age = data["Age"].mean()
print("Average age:", round(avg_age, 2))

# Filter out the students who have a grade above a certain threshold(consider the
threshold grade is 'B')
filtered_students = data[data["Grade"] <= "B"]
print("Students with a grade up to B")
print(filtered_students)
```

OUTPUT :

Average time

0.103 s

103.00 ms

Maximum time

0.149 s

149.00 ms

1 out of 1 shown test case(s) passed

1 out of 1 hidden test case(s) passed

Test case 1

149 ms

Debug

Expected output

Actual output

studentdata.txt

studentdata.txt

First five rows:

First five rows:

...Name...Age...Grade...

...Name...Age...Grade...

0..John...25...A

0..John...25...A

1..Alin...22...B

1..Alin...22...B

2..Emma...24...A

2..Emma...24...A

3..Mike...23...C

3..Mike...23...C

4..Sony...21...B

4..Sony...21...B

Average age: 23.29

Average age: 23.29

Students with a grade up to B

Students with a grade up to B

...Name...Age...Grade...

...Name...Age...Grade...

0..John...25...A

0..John...25...A

1..Alin...22...B

1..Alin...22...B

2..Emma...24...A

2..Emma...24...A

4..Sony...21...B

4..Sony...21...B

5..Ania...26...A

5..Ania...26...A

4.2.1. Month with the Highest Total Sales

PROBLEM STATEMENT :

Write a Python program that takes the file name of a CSV file as input, reads the data, and performs the following operations:

- The CSV file contains the columns: Date, Product, Quantity, Price, and City.
- Group the data by Month and calculate the total sales for each month.
- Find the month with the highest total sales and display it.
- Also, display the total sales for the best month.

Sample Data:

Date,Product,Quantity,Price,City

2025-01-01,Product A,5,20,New York

2025-01-01,Product B,3,15,Los Angeles

2025-01-02,Product A,7,20,New York

2025-01-02,Product C,4,30,Chicago

2025-01-03,Product B,2,15,Chicago

2025-01-03,Product A,8,20,Los Angeles

2025-01-04,Product C,6,30,New York

2025-01-04,Product B,5,15,Los Angeles

2025-01-05,Product A,3,20,Chicago

2025-01-05,Product C,10,30,Los Angeles

Note:

The data cannot be displayed in the file. You can refer to the sample data provided for insights.

CODE :

```
import pandas as pd

# Prompt the user for the file name
file_name = input()

# Load the data
df = pd.read_csv(file_name)

df['Date'] = pd.to_datetime(df['Date'])

df['Month'] = df['Date'].dt.to_period('M').astype(str)

df['Sales'] = df['Quantity'] * df['Price']

monthly_sales = df.groupby('Month')['Sales'].sum()

best_month = monthly_sales.idxmax()
highest_sales = monthly_sales.max()

print(f"Best month: {best_month}")
print(f"Total sales: ${highest_sales:.2f}")
```

OUTPUT :

Average time

0.056 s

56.33 ms

Maximum time

0.112 s

112.00 ms

1 out of 1 shown test case(s) passed

2 out of 2 hidden test case(s) passed

Test case 1

112 ms

Debug

Expected output

Actual output

sales_data.csv

sales_data.csv

Best month: 2025-01

Best month: 2025-01

Total sales: \$1218.00

Total sales: \$1218.00

4.2.2. Best Selling Product

PROBLEM STATEMENT :

Write a Python program that takes the file name of a CSV file as input, reads the data, and performs the following operations:

- The CSV file contains the columns: Date, Product, Quantity, Price, and City.
- Find the product that sold the most in terms of quantity sold.
- Display the product that sold the most and the total quantity sold for that product.

Sample Data:

Date,Product,Quantity,Price,City

2025-01-01,Product A,5,20,New York

2025-01-01,Product B,3,15,Los Angeles

2025-01-02,Product A,7,20,New York

2025-01-02,Product C,4,30,Chicago

2025-01-03,Product B,2,15,Chicago

2025-01-03,Product A,8,20,Los Angeles

2025-01-04,Product C,6,30,New York

2025-01-04,Product B,5,15,Los Angeles

2025-01-05,Product A,3,20,Chicago

2025-01-05,Product C,10,30,Los Angeles

Note:

The data cannot be displayed in the file. You can refer to the sample data provided for insights.

CODE :

```
import pandas as pd

# Prompt the user for the file name
file_name = input()

# Load the data
df = pd.read_csv(file_name)

product_sales = df.groupby("Product")["Quantity"].sum()

# Find the product with the highest total quantity sold
best_product = product_sales.idxmax()
highest_quantity = product_sales.max()

# Display the result
print(f"Best selling product: {best_product}")
print(f"Total quantity sold: {highest_quantity}")
```

OUTPUT :

Average time
0.035 s
35.00 ms

Maximum time
0.072 s
72.00 ms

1 out of 1 shown test case(s) passed

2 out of 2 hidden test case(s) passed

Test case 1    

Expected output

Actual output

sales_data.csv

Best selling product: Product A

Total quantity sold: 23

sales_data.csv

Best selling product: Product A

Total quantity sold: 23

4.2.3. City that Sold the Most Products

PROBLEM STATEMENT :

Write a Python program that takes the file name of a CSV file as input, reads the data, and performs the following operations:

- The CSV file contains the columns: Date, Product, Quantity, Price, and City.
- Group the data by City and calculate the total quantity of products sold for each city.
- Find the city that sold the most products (based on the total quantity sold).

Sample Data:

Date,Product,Quantity,Price,City

2025-01-01,Product A,5,20,New York

2025-01-01,Product B,3,15,Los Angeles

2025-01-02,Product A,7,20,New York

2025-01-02,Product C,4,30,Chicago

2025-01-03,Product B,2,15,Chicago

2025-01-03,Product A,8,20,Los Angeles

2025-01-04,Product C,6,30,New York

2025-01-04,Product B,5,15,Los Angeles

2025-01-05,Product A,3,20,Chicago

2025-01-05,Product C,10,30,Los Angeles

Note:

The data cannot be displayed in the file. You can refer to the sample data provided for insights.

CODE :

```
import pandas as pd

# Prompt the user for the file name
file_name = input()

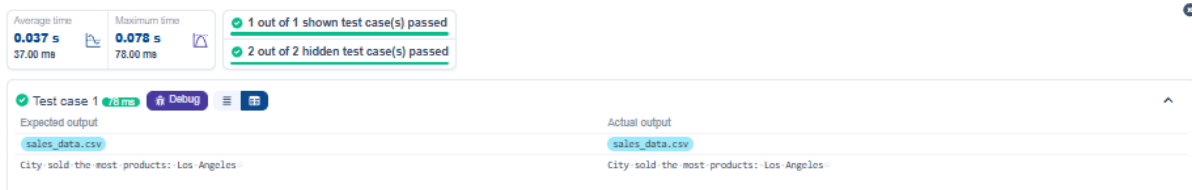
# Load the data
df = pd.read_csv(file_name)

# write the code..
city_sales = df.groupby("City")["Quantity"].sum()

best_city = city_sales.idxmax()

# Display the result
print(f"City sold the most products: {best_city}")
```

OUTPUT :



The screenshot displays a code execution interface. At the top, performance metrics are shown: Average time 0.037 s (37.00 ms) and Maximum time 0.078 s (78.00 ms). Test results indicate 1 out of 1 shown test case(s) passed and 2 out of 2 hidden test case(s) passed. Below this, a 'Test case 1' section shows the 'Expected output' and 'Actual output' for a file named 'sales_data.csv'. Both outputs are identical: 'City sold the most products: Los Angeles'.

Metric	Value
Average time	0.037 s (37.00 ms)
Maximum time	0.078 s (78.00 ms)

Test Results:

- 1 out of 1 shown test case(s) passed
- 2 out of 2 hidden test case(s) passed

Test case 1 (0.0 ms) [Debug]

Expected output	Actual output
City sold the most products: Los Angeles	City sold the most products: Los Angeles

4.2.4. Most Frequently Sold Product Pairs

PROBLEM STATEMENT :

Write a Python program that takes the file name of a CSV file as input, reads the data, and performs the following operations:

- The CSV file contains the following columns: Date, Product, Quantity, Price, and City.
- For each date, find all pairs of products that were sold together (i.e., two products sold on the same date).
- Output the product pair/s that was sold most frequently.

Sample Data:

Date,Product,Quantity,Price,City

2025-01-01,Product A,5,20,New York

2025-01-01,Product B,3,15,Los Angeles

2025-01-02,Product A,7,20,New York

2025-01-02,Product C,4,30,Chicago

2025-01-03,Product B,2,15,Chicago

2025-01-03,Product A,8,20,Los Angeles

2025-01-04,Product C,6,30,New York

2025-01-04,Product B,5,15,Los Angeles

2025-01-05,Product A,3,20,Chicago

2025-01-05,Product C,10,30,Los Angeles

Explanation:

Transactions:

- **2025-01-01:** Product A, Product B
- **2025-01-02:** Product A, Product C
- **2025-01-03:** Product B, Product A
- **2025-01-04:** Product C, Product B
- **2025-01-05:** Product A, Product C

Now, let's count how often the pairs of products appear together:

- **Product A and Product B:** Appear in transactions on 2025-01-01 and 2025-01-03.
- **Product A and Product C:** Appear in transactions on 2025-01-02 and 2025-01-05.
- **Product B and Product C:** Appears in transactions on 2025-01-04.

Most Frequent Product Combinations:

- **Product A and Product B** (2 times)
- **Product A and Product C** (2 times)

Note:

The data cannot be displayed in the file. You can refer to the sample data provided for insights.

CODE :

```
import pandas as pd

from itertools import combinations

from collections import Counter


# Prompt user to input the file name
file_name = input()


# Read data from the specified CSV file
df = pd.read_csv(file_name)


# write the code
grouped = df.groupby('Date')['Product'].apply(list)


# Generate all product pairs for each transaction
pairs_counter = Counter()

for products in grouped:
    pairs = combinations(sorted(products), 2) # Sort to ensure consistency
    pairs_counter.update(pairs)


# Find the most common product pairs
max_frequency = max(pairs_counter.values())

most_common_pairs = [(pair, freq) for pair, freq in pairs_counter.items() if freq ==
max_frequency]


# Display the most frequent product pairs
for (product1, product2), frequency in most_common_pairs:
    print(f"{product1} and {product2}: {frequency} times")
```

OUTPUT :

Average time

0.039 s

38.67 ms

Maximum time

0.078 s

78.00 ms

1 out of 1 shown test case(s) passed

2 out of 2 hidden test case(s) passed

Test case 1

78 ms

Debug

Expected output

Actual output

sales_data.csv

sales_data.csv

Product A and Product B: 2 times

Product A and Product B: 2 times

Product A and Product C: 2 times

Product A and Product C: 2 times

4.2.5. Titanic Dataset Analysis and Data Cleaning

PROBLEM STATEMENT :

You are provided with the Titanic dataset containing information about passengers on the Titanic. Your task is to write Python code to answer the following questions based on the dataset. For each question, perform necessary data cleaning, transformations, and calculations as required.

1. Display the first 5 rows of the dataset.
2. Display the last 5 rows of the dataset.
3. Get the shape of the dataset (number of rows and columns).
4. Get a summary of the dataset (using `.info()`).
5. Get basic statistics (mean, standard deviation, etc.) of the dataset using `.describe()`.
6. Check for missing values and display the count of missing values for each column.
7. Fill missing values in the 'Age' column with the median age.
8. Fill missing values in the 'Embarked' column with the most frequent value (`mode()`).
9. Drop the 'Cabin' column due to many missing values.
10. Create a new column, 'FamilySize' by adding the 'SibSp' and 'Parch' columns.

The Titanic dataset contains columns as shown below,

PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked

Sample Data:

PassengerId,Survived,Pclass,Name,Sex,Age,SibSp,Parch,Ticket,Fare,Cabin,Embarked

```
1,0,3,"Braund, Mr. Owen Harris",male,22,1,0,A/5 21171,7.25,,S
2,1,1,"Cumings, Mrs. John Bradley (Florence Briggs Thayer)",female,38,1,0,PC 17599,71.2833,C85,C
3,1,3,"Heikkinen, Miss. Laina",female,26,0,0,STON/O2. 3101282,7.925,,S
4,1,1,"Futrelle, Mrs. Jacques Heath (Lily May Peel)",female,35,1,0,113803,53.1,C123,S
5,0,3,"Allen, Mr. William Henry",male,35,0,0,373450,8.05,,S
6,0,3,"Moran, Mr. James",male,,0,0,330877,8.4583,,Q
7,0,1,"McCarthy, Mr. Timothy J",male,54,0,0,17463,51.8625,E46,S
8,0,3,"Palsson, Master. Gosta Leonard",male,2,3,1,349909,21.075,,S
9,1,3,"Johnson, Mrs. Oscar W (Elisabeth Vilhelmina Berg)",female,27,0,2,347742,11.1333,,S
10,1,2,"Nasser, Mrs. Nicholas (Adele Achem)",female,14,1,0,237736,30.0708,,C
```

Note: Refer to the visible test case for better reference.

CODE :

```
import pandas as pd
import numpy as np

# Load the Titanic dataset
data = pd.read_csv('Titanic-Dataset.csv')

# 1. Display the first 5 rows of the dataset
print(data.head())

# 2. Display the last 5 rows of the dataset
print(data.tail())

# 3. Get the shape of the dataset
print(data.shape)

# 4. Get a summary of the dataset (using info())
print(data.info())

# 5. Get basic statistics of the dataset
print(data.describe())

# 6. Check for missing values
print(data.isnull().sum())

# 7. Fill missing values in the 'Age' column with the median age
median_age = data['Age'].median()
data['Age'].fillna(median_age, inplace=True)
```

8. Fill missing values in the 'Embarked' column with the most frequent value (mode)

```
mode_embarked = data['Embarked'].mode()[0]
```

```
data['Embarked'].fillna(mode_embarked, inplace=True)
```

9. Drop the 'Cabin' column due to many missing values

```
data.drop('Cabin', axis=1, inplace=True)
```

10. Create a new column 'FamilySize' by adding 'SibSp' and 'Parch' columns

```
data['FamilySize'] = data['SibSp'] + data['Parch']
```

OUTPUT :

Average time
0.376 s
376.00 ms

Maximum time
0.376 s
376.00 ms

1 out of 1 shown test case(s) passed

Test case 1

Debug

Expected output

Actual output

PassengerId Survived Pclass Fare Cabin Embarked

0 1 0 3 7.2500 NaN S

1 2 1 1 71.2833 C85 C

2 3 1 3 7.9250 NaN S

3 4 1 1 53.1000 C123 S

4 5 0 3 8.0500 NaN S

[5 rows x 12 columns]

PassengerId Survived Pclass Fare Cabin Embarked

886 887 0 2 13.00 NaN S

887 888 1 1 30.00 B42 S

888 889 0 3 23.45 NaN S

889 890 1 1 30.00 C148 C

890 891 0 3 7.75 NaN Q

[5 rows x 12 columns]

(891, 12)

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 891 entries, 0 to 890

Data columns (total 12 columns):

0 Column Non-Null Count Dtype

0 PassengerId 891 non-null int64

1 Survived 891 non-null int64

2 Pclass 891 non-null int64

3 Name 891 non-null object

4 Sex 891 non-null object

5 Age 714 non-null float64

6 SibSp 891 non-null int64

7 Parch 891 non-null int64

8 Ticket 891 non-null object

9 Fare 891 non-null float64

10 Cabin 204 non-null object

11 Embarked 889 non-null object

dtypes: float64(2), int64(5), object(5)

memory usage: 66.2+ KB

None

PassengerId Survived Pclass SibSp Parch Fare

count 891.000000 891.000000 891.000000 891.000000 891.000000 891.000000

mean 446.000000 0.383838 2.308642 0.523008 0.381594 32.204208

std 257.353842 0.486592 0.836071 1.102743 0.806057 49.693429

min 1.000000 0.000000 1.000000 0.000000 0.000000 0.000000

25% 223.500000 0.000000 2.000000 0.000000 0.000000 7.910400

50% 446.000000 0.000000 3.000000 0.000000 0.000000 14.454200

75% 668.500000 1.000000 3.000000 1.000000 0.000000 31.000000

max 891.000000 1.000000 3.000000 8.000000 6.000000 512.329200

[8 rows x 7 columns]

PassengerId Survived Pclass Fare Cabin Embarked

0 1 0 3 7.2500 NaN S

1 2 1 1 71.2833 C85 C

2 3 1 3 7.9250 NaN S

3 4 1 1 53.1000 C123 S

4 5 0 3 8.0500 NaN S

[5 rows x 12 columns]

PassengerId Survived Pclass Fare Cabin Embarked

886 887 0 2 13.00 NaN S

887 888 1 1 30.00 B42 S

888 889 0 3 23.45 NaN S

889 890 1 1 30.00 C148 C

890 891 0 3 7.75 NaN Q

[5 rows x 12 columns]

(891, 12)

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 891 entries, 0 to 890

Data columns (total 12 columns):

0 Column Non-Null Count Dtype

0 PassengerId 891 non-null int64

1 Survived 891 non-null int64

2 Pclass 891 non-null int64

3 Name 891 non-null object

4 Sex 891 non-null object

5 Age 714 non-null float64

6 SibSp 891 non-null int64

7 Parch 891 non-null int64

8 Ticket 891 non-null object

9 Fare 891 non-null float64

10 Cabin 204 non-null object

11 Embarked 889 non-null object

dtypes: float64(2), int64(5), object(5)

memory usage: 66.2+ KB

None

PassengerId Survived Pclass SibSp Parch Fare

count 891.000000 891.000000 891.000000 891.000000 891.000000 891.000000

mean 446.000000 0.383838 2.308642 0.523008 0.381594 32.204208

std 257.353842 0.486592 0.836071 1.102743 0.806057 49.693429

min 1.000000 0.000000 1.000000 0.000000 0.000000 0.000000

25% 223.500000 0.000000 2.000000 0.000000 0.000000 7.910400

50% 446.000000 0.000000 3.000000 0.000000 0.000000 14.454200

75% 668.500000 1.000000 3.000000 1.000000 0.000000 31.000000

max 891.000000 1.000000 3.000000 8.000000 6.000000 512.329200

[8 rows x 7 columns]

PassengerId

Survived

Pclass

Fare

Cabin

Embarked

Age

SibSp

Parch

Ticket

Fare

Prev

Reset

Submit

Next

4.2.6. Titanic Dataset Analysis and Data Cleaning – 2

PROBLEM STATEMENT :

You are provided with the Titanic dataset containing information about passengers on the Titanic. Your task is to write Python code to answer the following questions based on the dataset.

1. Create a new column 'IsAlone' which is 1 if the passenger is alone (FamilySize = 0), otherwise 0.
2. Convert the 'Sex' column to numeric values (male: 0, female: 1).
3. One-hot encode the 'Embarked' column, dropping the first category.
4. Get the mean age of passengers.
5. Get the median fare of passengers.
6. Get the number of passengers by class.
7. Get the number of passengers by gender.
8. Get the number of passengers by survival status.
9. Calculate the survival rate of passengers.
10. Calculate the survival rate by gender.

The Titanic dataset contains columns as shown below,

Passenger Id	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked

Sample Data:

PassengerId,Survived,Pclass,Name,Sex,Age,SibSp,Parch,Ticket,Fare,Cabin,Embarked

1,0,3,"Braund, Mr. Owen Harris",male,22,1,0,A/5 21171,7.25,,S

2,1,1,"Cumings, Mrs. John Bradley (Florence Briggs Thayer)",female,38,1,0,PC
17599,71.2833,C85,C

3,1,3,"Heikkinen, Miss. Laina",female,26,0,0,STON/O2. 3101282,7.925,,S

4,1,1,"Futrelle, Mrs. Jacques Heath (Lily May Peel)",female,35,1,0,113803,53.1,C123,S

5,0,3,"Allen, Mr. William Henry",male,35,0,0,373450,8.05,,S
6,0,3,"Moran, Mr. James",male,,0,0,330877,8.4583,,Q
7,0,1,"McCarthy, Mr. Timothy J",male,54,0,0,17463,51.8625,E46,S
8,0,3,"Palsson, Master. Gosta Leonard",male,2,3,1,349909,21.075,,S
9,1,3,"Johnson, Mrs. Oscar W (Elisabeth Vilhelmina Berg)",female,27,0,2,347742,11.1333,,S
10,1,2,"Nasser, Mrs. Nicholas (Adele Achem)",female,14,1,0,237736,30.0708,,C

Note: Refer to the visible test case for better reference.

CODE :

```
import pandas as pd
import numpy as np

# Load the Titanic dataset
data = pd.read_csv('Titanic-Dataset.csv')
data['FamilySize'] = data['SibSp'] + data['Parch']

data['IsAlone'] = np.where(data['FamilySize'] == 0, 1, 0)
# 1. Create a new column 'IsAlone' which is 1 if the passenger is alone

# 2. Convert 'Sex' column to numeric values (male: 0, female: 1)
data['Sex'] = data['Sex'].map({'male': 0, 'female': 1})

# 3. One-hot encode the 'Embarked' column, dropping the first category
data = pd.get_dummies(data, columns=['Embarked'], drop_first=True)

# 4. Get the mean age of passengers
```

```
mean_age = data['Age'].mean()
print(mean_age)
```

5. Get the median fare of passengers

```
median_fare = data['Fare'].median()
print(median_fare)
```

6. Get the number of passengers by class

```
print(data['Pclass'].value_counts())
```

7. Get the number of passengers by gender

```
print(data['Sex'].value_counts().sort_index())
```

8. Get the number of passengers by survival status

```
print(data['Survived'].value_counts().sort_index())
```

9. Calculate the survival rate of passengers

```
survival_rate = data['Survived'].mean()
print(survival_rate)
```

10. Calculate the survival rate by gender

```
print(data.groupby('Sex')['Survived'].mean())
```

OUTPUT :

Average time
0.061 s
61.00 ms

Maximum time
0.061 s
61.00 ms

1 out of 1 shown test case(s) passed

Test case 1 61 ms Debug

Expected output	Actual output
29.69911764705882	29.69911764705882
14.4542	14.4542
3...491	3...491
1...216	1...216
2...184	2...184
Name: Pclass, dtype: int64	Name: Pclass, dtype: int64
0...577	0...577
1...314	1...314
Name: Sex, dtype: int64	Name: Sex, dtype: int64
0...549	0...549
1...342	1...342
Name: Survived, dtype: int64	Name: Survived, dtype: int64
0.3838383838383838	0.3838383838383838
Sex	Sex
0...0.188908	0...0.188908
1...0.742038	1...0.742038
Name: Survived, dtype: float64	Name: Survived, dtype: float64

Terminal

Test cases

< Prev

Reset

Submit

Next >

4.2.7. Titanic Dataset Analysis and Data Cleaning – 3

PROBLEM STATEMENT :

You are provided with the Titanic dataset containing information about passengers on the Titanic. Your task is to write Python code to answer the following questions based on the dataset.

1. Calculate the survival rate by class.
2. Calculate the survival rate by embarkation location (Embarked_S).
3. Calculate the survival rate by family size (FamilySize).
4. Calculate the survival rate by being alone (IsAlone).
5. Get the average fare by passenger class (Pclass).
6. Get the average age by passenger class (Pclass).
7. Get the average age by survival status (Survived).
8. Get the average fare by survival status (Survived).
9. Get the number of survivors by class (Pclass).
10. Get the number of non-survivors by class (Pclass).

The Titanic dataset contains columns as shown below,

Passenger Id	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked

Sample Data:

PassengerId,Survived,Pclass,Name,Sex,Age,SibSp,Parch,Ticket,Fare,Cabin,Embarked

1,0,3,"Braund, Mr. Owen Harris",male,22,1,0,A/5 21171,7.25,,S

2,1,1,"Cumings, Mrs. John Bradley (Florence Briggs Thayer)",female,38,1,0,PC
17599,71.2833,C85,C

3,1,3,"Heikkinen, Miss. Laina",female,26,0,0,STON/O2. 3101282,7.925,,S

4,1,1,"Futrelle, Mrs. Jacques Heath (Lily May Peel)",female,35,1,0,113803,53.1,C123,S
5,0,3,"Allen, Mr. William Henry",male,35,0,0,373450,8.05,,S
6,0,3,"Moran, Mr. James",male,,0,0,330877,8.4583,,Q
7,0,1,"McCarthy, Mr. Timothy J",male,54,0,0,17463,51.8625,E46,S
8,0,3,"Palsson, Master. Gosta Leonard",male,2,3,1,349909,21.075,,S
9,1,3,"Johnson, Mrs. Oscar W (Elisabeth Vilhelmina Berg)",female,27,0,2,347742,11.1333,,S
10,1,2,"Nasser, Mrs. Nicholas (Adele Achem)",female,14,1,0,237736,30.0708,,C

Note: Refer to the visible test case for better reference.

CODE :

```
import pandas as pd
import numpy as np

# Load the Titanic dataset
data = pd.read_csv('Titanic-Dataset.csv')
data['FamilySize'] = data['SibSp'] + data['Parch']
data['IsAlone'] = np.where(data['FamilySize'] > 0, 0, 1)
data = pd.get_dummies(data, columns=['Embarked'], drop_first=True)

# 1. Calculate the survival rate by class
print(data.groupby('Pclass')['Survived'].mean())

# 2. Calculate the survival rate by embarked location (Embarked_S)
print(data.groupby('Embarked_S')['Survived'].mean())
```

3. Calculate the survival rate by family size

```
print(data.groupby('FamilySize')['Survived'].mean())
```

4. Calculate the survival rate by being alone

```
print(data.groupby('IsAlone')['Survived'].mean())
```

5. Get the average fare by class

```
print(data.groupby('Pclass')['Fare'].mean())
```

6. Get the average age by class

```
print(data.groupby('Pclass')['Age'].mean())
```

7. Get the average age by survival status

```
print(data.groupby('Survived')['Age'].mean())
```

8. Get the average fare by survival status

```
print(data.groupby('Survived')['Fare'].mean())
```

9. Get the number of survivors by class

```
print(data[data['Survived'] == 1]['Pclass'].value_counts())
```

10. Get the number of non-survivors by class

```
print(data[data['Survived'] == 0]['Pclass'].value_counts())
```

OUTPUT :

Explorer

Average time
0.064 s
64.00 ms

Maximum time
0.064 s
64.00 ms

1 out of 1 shown test case(s) passed

Debugger

Test case 1 64 ms Debug

Expected output

Actual output

Pclass	Pclass
1...0.629630	1...0.629630
2...0.472826	2...0.472826
3...0.242363	3...0.242363
Name: Survived, dtype: float64	Name: Survived, dtype: float64
Embarked_S	Embarked_S
0...0.506073	0...0.506073
1...0.336957	1...0.336957
Name: Survived, dtype: float64	Name: Survived, dtype: float64
FamilySize	FamilySize
0...0.303538	0...0.303538
1...0.552795	1...0.552795
2...0.578431	2...0.578431
3...0.724138	3...0.724138
4...0.200000	4...0.200000
5...0.136364	5...0.136364
6...0.333333	6...0.333333
7...0.000000	7...0.000000
10...0.000000	10...0.000000
Name: Survived, dtype: float64	Name: Survived, dtype: float64
IsAlone	IsAlone
0...0.505650	0...0.505650
1...0.303538	1...0.303538
Name: Survived, dtype: float64	Name: Survived, dtype: float64
Pclass	Pclass
1...84.154687	1...84.154687
2...20.662183	2...20.662183
3...13.675550	3...13.675550
Name: Fare, dtype: float64	Name: Fare, dtype: float64

< Prev

Reset

Submit

Next >

4.2.8. Titanic Dataset Analysis and Data Cleaning – 4

PROBLEM STATEMENT :

You are provided with the Titanic dataset containing information about passengers on the Titanic. Your task is to write Python code to answer the following questions based on the dataset.

1. Get the number of survivors by gender (Sex).
2. Get the number of non-survivors by gender (Sex).
3. Get the number of survivors by embarkation location (Embarked_S).
4. Get the number of non-survivors by embarkation location (Embarked_S).
5. Calculate the percentage of children (Age < 18) who survived.
6. Calculate the percentage of adults (Age >= 18) who survived.
7. Get the median age of survivors.
8. Get the median age of non-survivors.
9. Get the median fare of survivors.
10. Get the median fare of non-survivors.

The Titanic dataset contains columns as shown below,

Passenger Id	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked

Sample Data:

PassengerId,Survived,Pclass,Name,Sex,Age,SibSp,Parch,Ticket,Fare,Cabin,Embarked

1,0,3,"Braund, Mr. Owen Harris",male,22,1,0,A/5 21171,7.25,,S

2,1,1,"Cumings, Mrs. John Bradley (Florence Briggs Thayer)",female,38,1,0,PC
17599,71.2833,C85,C

3,1,3,"Heikkinen, Miss. Laina",female,26,0,0,STON/O2. 3101282,7.925,,S

4,1,1,"Futrelle, Mrs. Jacques Heath (Lily May Peel)",female,35,1,0,113803,53.1,C123,S
5,0,3,"Allen, Mr. William Henry",male,35,0,0,373450,8.05,,S
6,0,3,"Moran, Mr. James",male,,0,0,330877,8.4583,,Q
7,0,1,"McCarthy, Mr. Timothy J",male,54,0,0,17463,51.8625,E46,S
8,0,3,"Palsson, Master. Gosta Leonard",male,2,3,1,349909,21.075,,S
9,1,3,"Johnson, Mrs. Oscar W (Elisabeth Vilhelmina Berg)",female,27,0,2,347742,11.1333,,S
10,1,2,"Nasser, Mrs. Nicholas (Adele Achem)",female,14,1,0,237736,30.0708,,C

Note: Refer to the visible test case for better reference.

CODE :

```
import pandas as pd
import numpy as np

# Load the Titanic dataset
data = pd.read_csv('Titanic-Dataset.csv')
data = pd.get_dummies(data, columns=['Embarked'], drop_first=True)

# 1. Get the number of survivors by gender (Sex)
print(data[data['Survived'] == 1]['Sex'].value_counts())

# 2. Get the number of non-survivors by gender (Sex)
print(data[data['Survived'] == 0]['Sex'].value_counts())
```

3. Get the number of survivors by embarkation location (Embarked_S)

```
print(data[data['Survived'] == 1]['Embarked_S'].value_counts())
```

4. Get the number of non-survivors by embarkation location (Embarked_S)

```
print(data[data['Survived'] == 0]['Embarked_S'].value_counts())
```

5. Calculate the percentage of children (Age < 18) who survived

```
children = data[data['Age'] < 18]
```

```
print(children['Survived'].mean())
```

6. Calculate the percentage of adults (Age >= 18) who survived

```
adults = data[data['Age'] >= 18]
```

```
print(adults['Survived'].mean())
```

7. Get the median age of survivors

```
print(data[data['Survived'] == 1]['Age'].median())
```

8. Get the median age of non-survivors

```
print(data[data['Survived'] == 0]['Age'].median())
```

9. Get the median fare of survivors

```
print(data[data['Survived'] == 1]['Fare'].median())
```

10. Get the median fare of non-survivors

```
print(data[data['Survived'] == 0]['Fare'].median())
```

OUTPUT :

Average time
0.054 s
54.00 ms

Maximum time
0.054 s
54.00 ms

1 out of 1 shown test case(s) passed

Test case 154 msDebug

Expected output

Actual output

female....233	female....233
male.....109	male.....109
Name:Sex, dtype:int64	Name:Sex, dtype:int64
male.....468	male.....468
female....81	female....81
Name:Sex, dtype:int64	Name:Sex, dtype:int64
1....217	1....217
0....125	0....125
Name:Embarked_S, dtype:int64	Name:Embarked_S, dtype:int64
1....427	1....427
0....122	0....122
Name:Embarked_S, dtype:int64	Name:Embarked_S, dtype:int64
0.5398230088495575	0.5398230088495575
0.3810316139767055	0.3810316139767055
28.0	28.0
28.0	28.0
26.0	26.0
10.5	10.5

< Prev

Reset

Submit

Next >